

Competencia de Programación - Estructuras de Datos

2026-2S

Este set de problemas tiene 4 problemas, ennumerados de la página 1 a la 6

12 de agosto de 2026

Facultad de Ciencias Naturales e Ingeniería
Universidad Jorge Tadeo Lozano

Todos los problemas son de autoría por Ludwig Alvarado o adaptados de
<https://codeforces.com>

Información General

Salvo que se indique lo contrario, las siguientes condiciones aplican a todos los problemas.

Entrada

1. La entrada se lee desde la entrada estándar.
2. Existe múltiples casos de prueba.
3. Los valores de una misma línea están separados por espacios.
4. No hay datos adicionales en la entrada.

Salida

1. La salida se escribe en la salida estándar.
2. Debe imprimirse exactamente el resultado solicitado, sin información adicional.

Material y herramientas permitidas

1. Se permite utilizar únicamente material de consulta **impreso**, como libros, apuntes y hojas de referencia.
2. También se permite consultar exclusivamente:
 - Python: <https://docs.python.org/>
 - Java: <https://docs.oracle.com/en/java/>
 - C/C++: <https://cppreference.net/>
3. No está permitido consultar ninguna otra página web, utilizar buscadores, foros, blogs, repositorios, sitios de soluciones ni fuentes externas.
4. No se permite el uso de celulares, relojes inteligentes, audífonos, tabletas u otros dispositivos personales. Estos deberán permanecer **guardados en la maleta durante toda la sesión**.
5. Está prohibido utilizar LLM, asistentes de inteligencia artificial o herramientas de generación automática de código, incluyendo ChatGPT, GitHub Copilot, Gemini, Claude o equivalentes.
6. El único IDE permitido es **USACO Guide IDE**, disponible en <https://ide.usaco.guide/>. Los lenguajes permitidos son **C/C++**, **Java** y **Python**.

Comunicación y colaboración

1. Los integrantes del grupo pueden discutir problemas, diseñar algoritmos, revisar código y tomar decisiones conjuntamente.
2. Está prohibido recibir o proporcionar ayuda a otros grupos, así como compartir código, algoritmos, soluciones o fragmentos de código.
3. Las consultas al profesor se limitarán a dudas sobre la **lectura o interpretación de los datos de entrada**. No se proporcionarán pistas sobre algoritmos, estrategias, complejidad o soluciones, ni se revisarán o validarán códigos.
4. No está permitido comunicarse con personas externas al grupo para obtener ayuda, ni observar o copiar información de otros grupos.

Problem A – Sandía

(Original en Codeforces: Watermelon)

En un caluroso día de verano, Pete y su amigo Billy decidieron comprar una sandía. En su opinión, eligieron la más grande y madura. Después de eso, pesaron la sandía y la báscula indicó que pesaba w kilogramos. Regresaron corriendo a casa, muertos de sed, y decidieron dividir la sandía. Sin embargo, se enfrentaron a un problema difícil.

Pete y Billy son grandes aficionados a los números pares, por lo que quieren dividir la sandía de tal manera que cada una de las dos partes pese un número par de kilogramos. Al mismo tiempo, no es obligatorio que las partes sean iguales. Los niños están extremadamente cansados y quieren comenzar a comer lo antes posible, por lo que debes ayudarlos a determinar si pueden dividir la sandía de la manera que desean.

Por supuesto, cada uno de ellos debe recibir una parte de peso positivo.

Entrada

La primera y única línea de entrada contiene un número entero w ($1 \leq w \leq 100$), que representa el peso de la sandía comprada por los niños, en kilogramos.

Salida

Imprime YES si los niños pueden dividir la sandía en dos partes, cada una con un peso par de kilogramos. En caso contrario, imprime NO.

Prueba #1

Entrada	Salida
8	YES

Explicación

Por ejemplo, los niños pueden dividir la sandía en dos partes de 2 y 6 kilogramos, respectivamente. Otra posibilidad es dividirla en dos partes de 4 y 4 kilogramos.

Problem B – Cabito Organizadito

Cabito (figura 1 a la izquierda) es la mascota de la Universidad Jorge Tadeo Lozano. Él es el sucesor de Cabo (figura 1 a la derecha, lo vamos a llamar Don Cabo), un perrito muy querido por toda la comunidad Tadeísta (al igual que Cabito) hace muchos años. . . Cabito llegó como un compañero de Don Cabo y es el que hoy en día conocemos en la Tadeo. Estos dos amados perritos en el corto tiempo que estuvieron juntos se pasaron algunas *mañas*, en especial Don Cabo (como gran mentor) a Cabito.



Figura 1: Cabito y Cabo. Autor de la foto: Universidad Jorge Tadeo Lozano

Entre una de las muchas *mañas*, está la comida. . . Don Cabo en un día de enseñanzas a Cabito le dijo: “*guau guau, guauuu guau guau woof, woof guau woofwoof. . .*” En español esto quiere decir “Cabito, cuando tengas hambre, pídele comida a esos ingenuos estudiantes. . .” Siguiendo sus aprendizajes, Cabito mira con sus tiernos ojos a los estudiantes en busca de comida y ellos le dan (**¡Nunca le debes dar comida a Cabito!**).

Cada vez que alguien le da comida a Cabito él se va a su *guarida secreta* y coloca la comida que le han dado en el suelo y organiza las unidades de comida por filas. Cabito sigue el siguiente patrón; en la primera fila coloca 1 unidad de comida, en la segunda fila coloca 2 unidades de comida, y así sucesivamente, entonces en la fila i Cabito coloca i unidades de comida.

Al final de cada día Cabito quiere saber cuántas filas de comida puede organizar y tu misión es escribir un programa para ayudarlo.

Entrada

Cabito te va a dar un número entero ($1 \leq n \leq 100$), la cantidad total de unidades de comida que logró recolectar a lo largo del día. Recuerda que Cabito organiza su comida por filas y en la primera fila debe haber 1 unidad de comida, en la segunda fila 2 unidades de comida, en la tercera fila 3 unidades de comida y así sucesivamente. . .

Salida

Lo que tu programa debe hacer es imprimir un número entero, la cantidad de filas que Cabito puede formar con el n dado. Si Cabito te da $n = 3$ (tres unidades de comida), en la primera fila hay una unidad de comida y en la segunda dos. Por lo tanto, lo que debes imprimir es 2 (porque hay 2 filas en total). Sin embargo, hay un detalle extra, en la fila i **deben haber mínimo** i unidades de comida. Es decir, en el caso de que Cabito te de $n = 4$ la comida se debe organizar así; en la primera fila una unidad de comida, en la segunda fila **3** unidades de comida. Esto se debe a que para tener una tercera fila deben haber **mínimo** 3 unidades de comida, por lo tanto, para el caso $n = 4$ tu programa debe imprimir 2.

Prueba #1

Entrada	Salida
3	2

Prueba #2

Entrada	Salida
6	3

Prueba #3

Entrada	Salida
4	2

Explicación

En la primera prueba, Cabito reparte 3 unidades: pone 1 en la primera fila y 2 en la segunda, usando en total 2 filas.

En la tercera prueba, con 4 unidades: coloca 1 en la primera fila y 3 en la segunda. No alcanza para una tercera (que requeriría 6 unidades en total, vea prueba 2), así que también se usan 2 filas.

Problem C – Votación Hexagonal

Dentro de la Universidad Tecnológica de Algoritmos y Desarrollo Optimizado (UTADEO), se están organizando las próximas elecciones para representantes estudiantiles al consejo directivo. Debido a la polémica que hubo el año pasado por un posible fraude, la universidad decidió seleccionar 6 puntos físicos vigilados para que los estudiantes puedan ir a votar que se pueden ver en la figura 2.

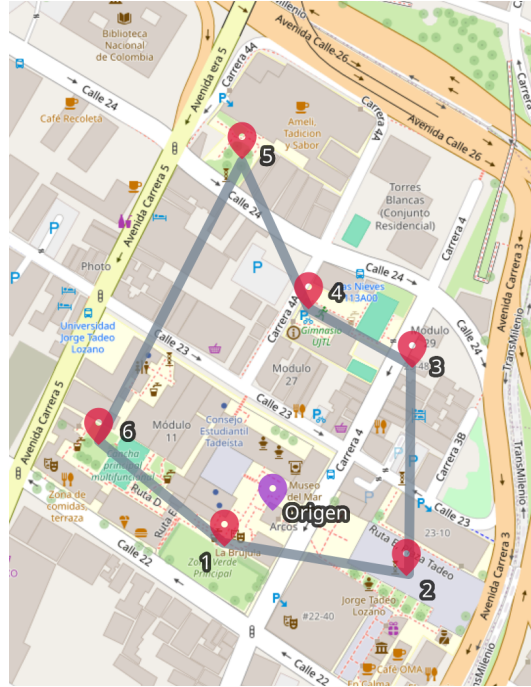


Figura 2: Puestos de votación disponibles, datos cartográficos de OpenStreetMap, edición por autoría propia.

Debido a que este año la universidad asignó un presupuesto mucho mayor, te contrataron a ti como programador para mostrar los resultados de las elecciones, distancia del puesto de votación más lejano al origen, y el Índice Energético de Votación (IEV) definido como:

$$IEV = \lceil \sqrt[3]{V} \rceil!$$

Donde V es la cantidad de votos totales en una mesa de votación. Para este año se postularon 3 estudiantes para ser el nuevo representante al consejo directivo de la universidad, ellos son; Andrea (A), Bobby (B) y Claudia (C).

Entrada

Vas a recibir 6 líneas de datos, en cada línea vas a encontrar las coordenadas (x, y) , $(-1000 < x, y < 1000)$ de cada puesto de votación con respecto al origen $(0, 0)$ seguido de la cantidad de votos que tuvo Andrea, Bobby y Claudia en esa mesa de votación, $(1 \leq A, B, C \leq 500)$.

Salida

Se deben imprimir el total de votos de todas las mesas de votación, la cantidad de votos que tuvo cada candidato en todas las mesas, el porcentaje de votos de cada candidato, la mesa más lejana al origen y el IEV del puesto de votación con más votos recogidos.

Se debe seguir el siguiente formato:

Total Votos: #numeroVotos

Votos Andrea: #votos #porcentaje

Votos Bobby: #votos #porcentaje
Votos Claudia: #votos #porcentaje
Mesa más lejana: #numeroMesa #distancia
IEV: #numeroMesa #valorIEV

Prueba #1

Entrada	Salida
-1 -1 78 50 90	Total Votos: 2248
5 -1 100 90 115	Votos Andrea: 798 35.50
3 7 20 15 10	Votos Bobby: 755 33.59
0 8 40 55 30	Votos Claudia: 695 30.92
-2 11 470 450 370	Mesa mas lejana: 5 11.18
-7 2 90 95 80	IEV: 5 39916800

Problem D – Los números de Cabito

En la Universidad Tecnológica de Algoritmos y Desarrollo Optimizado (UTADEO), Cabito ha decidido que quiere aprender programación para poder ayudar a los estudiantes en sus clases. Como todavía está aprendiendo, su profesor decidió comenzar con un ejercicio sencillo sobre números pares e impares.

Un día, Cabito encontró una lista de números que algunos estudiantes habían dejado en un computador de la universidad. Como Cabito es un perrito muy organizado, decidió revisar cada uno de los números de la lista y clasificarlos.

Sin embargo, Cabito todavía no sabe contar muy bien. Por eso, necesita tu ayuda para determinar cuántos números pares y cuántos números impares hay en la lista.

Entrada

La primera línea contiene un entero N ($1 \leq N \leq 1000$), que representa la cantidad de números que hay en la lista.

La segunda línea contiene N números enteros $a_1, a_2, \dots, a_N$ ($-10^9 \leq a_i \leq 10^9$).

Salida

Debes imprimir dos números enteros: primero, la cantidad de números pares de la lista, y segundo, la cantidad de números impares.

Los dos números deben estar separados por un espacio.

Prueba #1

Entrada	Salida
5 1 2 3 4 5	2 3

Prueba #2

Entrada	Salida
8 10 7 4 9 12 15 20 21	4 4

Prueba #3

Entrada	Salida
6 -2 -5 0 8 11 13	3 3

Explicación

En la primera prueba, la lista contiene los números 1, 2, 3, 4, 5. Los números pares son 2 y 4, por lo que hay 2 números pares. Los números impares son 1, 3 y 5, por lo que hay 3 números impares.

En la segunda prueba hay cuatro números pares: 10, 4, 12 y 20. Los cuatro números restantes son impares.

En la tercera prueba, los números pares son -2, 0 y 8, mientras que -5, 11 y 13 son impares. Por lo tanto, la respuesta es 3 3.